

CSE420 · COMPILER DESIGN

LR(0) Automaton

LR(0) Items · Augmented Grammar · CLOSURE · GOTO · Canonical
Collection · DFA Construction · LR Parsing Algorithm

Prepared by Azmari Sultana

Table of Contents

1. Introduction to LR(k) Parsing	3
2. LR(0) Items and the Dot Notation	3
3. Augmented Grammar	4
4. The CLOSURE Function	5
5. The GOTO Function	6
6. Canonical Collection & DFA Construction	6
7. Worked Example: Expression Grammar (I0–I11)	7
8. Parsing With the Automaton: Stack Trace	8
9. SLR(1) Parsing Table (Same Grammar)	9
10. Simulation: If / While Statement Grammar	10
11. Simulation: $S \rightarrow SS$ Grammar (Not LR(0))	11
12. Simulation: if–E–do–S–end Grammar	12
13. Simulation: $S \rightarrow A D$ Grammar (Shift/Reduce Conflict)	13
14. Quiz Practice Examples	15
14.1 Declaration Grammar: $s \rightarrow a c$	15
14.2 Statement Grammar: $\text{expr} \rightarrow \text{while}(\text{cond}) \mid \text{epsilon} \mid \text{decl}$	16
14.3 Ambiguous Grammar: $S \rightarrow SSS^+ \mid SS^* \mid a \mid == \mid e$	16
14.4 Record Grammar: $p_t \rightarrow \text{record} \{ \text{def} \} \mid \epsilon$	17

1. Introduction to LR(k) Parsing

LR(k) parsing is a **bottom-up** parsing technique.

- **L** → scans input from Left to right.
- **R** → builds a Rightmost derivation, but in reverse (this is what "bottom-up" means).
- **k** → number of lookahead symbols used to make a parsing decision.

Only $k = 0$ and $k = 1$ matter in practice. If k is not written, $k = 1$ is assumed. This chapter studies **LR(0)**: $k = 0$, so no lookahead is used — decisions come purely from the items in the current state.

LR grammar: a grammar for which an LR parsing table can be built (using SLR, canonical-LR, or LALR).

Why LR Parsers Are Attractive

- Can recognise virtually all programming-language constructs written as CFGs.
- Most general non-backtracking shift-reduce method known, yet as efficient as simpler shift-reduce methods.
- Detects a syntax error as early as possible during a left-to-right scan.
- LR grammars are a proper superset of LL grammars — LR decides *after* seeing the whole right side (with k lookahead); LL must guess the production from only the first k symbols still to come.
- Main drawback: too much work to build an LR parser by hand for a real grammar, so parser generators (e.g. Yacc) are used.

Exam Tip: Always state what L, R, k stand for — guaranteed short-question marks. If asked why LR > LL in power, answer in terms of *when* lookahead is used.

Q: What does LR(k) stand for?

A: L = left-to-right scan, R = rightmost derivation in reverse, k = lookahead symbols used.

Q: Why are LR parsers preferred over other shift-reduce parsers?

A: Most general non-backtracking method, yet efficient; detects errors earliest; parses a strictly larger class of grammars than LL.

Q: How is the class of LR grammars related to LL grammars?

A: Every LL grammar is LR, but not every LR grammar is LL — LR is a proper superset.

Remember: LR = Left-to-right scan + Rightmost derivation in reverse + k lookahead symbols. This chapter: $k = 0$, no lookahead at all.

2. LR(0) Items and the Dot Notation

An **LR(0) item** is a production with a dot ($\bullet$) placed at some position in its body. It shows how much of the production has been recognised so far, and what is still expected.

Example: $A \rightarrow XYZ$ generates exactly four items:

- $A \rightarrow \bullet XYZ$ — nothing seen yet.
- $A \rightarrow X \bullet YZ$ — X seen, hope to see YZ.
- $A \rightarrow XY \bullet Z$ — X, Y seen, only Z remains.
- $A \rightarrow XYZ \bullet$ — entire body seen; ready to reduce.

Special case: $A \rightarrow \epsilon$ generates only **one** item: $A \rightarrow \bullet$ (the dot has nowhere else to go).

Key Points

- The dot is only a progress marker — not a grammar symbol.
- A body of n symbols generates exactly $n + 1$ items.
- **Initial item:** dot at extreme left. **Final item:** dot at extreme right — signals a possible reduction.

Q: How many items does $A \rightarrow XYZ$ generate?

A: Four: $A \rightarrow \bullet XYZ$, $A \rightarrow X \bullet YZ$, $A \rightarrow XY \bullet Z$, $A \rightarrow XYZ \bullet$.

Q: How many items does $A \rightarrow \epsilon$ generate?

A: Exactly one: $A \rightarrow \bullet$.

Q: Is the dot a grammar symbol?

A: No — it is only a bookkeeping marker used by the parser.

Common Mistake: Writing only 3 items for a 3-symbol body (forgetting the dot-at-front item). Treating $A \rightarrow \epsilon$ as zero items instead of exactly one.

Remember: Item = production + dot position. n symbols $\rightarrow n+1$ items. ϵ -production $\rightarrow$ exactly 1 item.

3. Augmented Grammar

If G has start symbol S , the **augmented grammar** G' is G plus a brand-new start symbol S' and exactly one new production:

$S' \rightarrow S$

This tells the parser exactly when to accept: acceptance happens if and only if the parser is about to reduce by $S' \rightarrow S$ (i.e. the whole input has collapsed back to S).

Key Points

- Only ONE production is added. Nothing else changes.
- S' must be a brand-new symbol.
- The starting item set I_0 is always $\text{CLOSURE}(\{[S' \rightarrow \bullet S]\})$.
- Without augmentation, the parser might reduce back to S in the middle of parsing something larger (if S appears on a right side elsewhere), with no clear "stop" signal.

Exam Tip: Every simulation problem starts with "Step 1: Augment the Grammar" — never skip writing $S' \rightarrow S$ explicitly, and renumber it as production 0.

Q: What is an augmented grammar?

A: G plus a new start symbol S' and a new production $S' \rightarrow S$.

Q: What is the start item of the LR(0) automaton?

A: $S' \rightarrow \bullet S$ (before closure).

Q: What goes wrong without augmentation?

A: The parser could reduce down to S before the whole input is read (if S occurs on some right side), so there'd be no unambiguous "accept" point.

Remember: Augmentation = original grammar + exactly one new rule $S' \rightarrow S$. Always Step 1.

4. The CLOSURE Function

If I is a set of items, $CLOSURE(I)$ is built using two rules:

- **Rule 1:** Every item already in I is in $CLOSURE(I)$.
- **Rule 2:** If $A \rightarrow \alpha \bullet B \beta$ is in $CLOSURE(I)$, and $B \rightarrow \gamma$ is a production, add $B \rightarrow \bullet \gamma$ (if not already present). Repeat until nothing new is added.

Intuitively: whenever the dot sits right before a non-terminal B , the parser might next need to recognise a string derivable from B — so every way B can start must be added.

Worked Example — Expression Grammar

Grammar: $E' \rightarrow E \mid E \rightarrow E+T \mid E \rightarrow T \mid T \rightarrow T * F \mid T \rightarrow F \mid F \rightarrow (E) \mid F \rightarrow id$

Start with $I = \{[E' \rightarrow \bullet E]\}$. Closure chases $E \rightarrow T \rightarrow F$:

$I_0 = CLOSURE(I) = \{ E' \rightarrow \bullet E, E \rightarrow \bullet E+T, E \rightarrow \bullet T, T \rightarrow \bullet T * F, T \rightarrow \bullet F, F \rightarrow \bullet (E), F \rightarrow \bullet id \}$

Pseudocode

```
SetOfItems CLOSURE(I) {
  J = I;
  repeat
    for (each item A -> a.Bb in J)
      for (each production B -> g of G)
        if (B -> .g is not in J)
          add B -> .g to J;
  until no more items are added to J on one round;
  return J;
}
```

Key Points

- CLOSURE only adds items whose dot is at the extreme left (nonkernel items).
- Must be applied repeatedly (fixed point) — one pass is often not enough.
- **Kernel items** = $S' \rightarrow \bullet S$, plus every item whose dot is NOT at the left end. **Nonkernel items** = every item with dot at the left end (except $S' \rightarrow \bullet S$).
- Because nonkernel items can always be regenerated, a state can be stored using only its kernel items.

Q: Define CLOSURE(I).

A: Items of I , plus — whenever $A \rightarrow \alpha \bullet B \beta$ is present and $B \rightarrow \gamma$ is a production — add $B \rightarrow \bullet \gamma$, repeated until nothing new is added.

Q: What is a kernel item vs a nonkernel item?

A: Kernel = $S' \rightarrow \bullet S$ plus items with dot not at left end. Nonkernel = items with dot at left end (except $S' \rightarrow \bullet S$).

Q: Does CLOSURE ever remove items?

A: No — purely additive.

Common Mistake: Stopping after one pass instead of repeating to a fixed point; forgetting to close an item set before calling it a "state".

5. The GOTO Function

$\text{GOTO}(I, X)$, where X is any grammar symbol, is the CLOSURE of all items $[A \rightarrow \alpha X \cdot \beta]$ such that $[A \rightarrow \alpha \cdot X \beta]$ is in I .

In words: find every item in I with X right after the dot, move the dot past X , then take CLOSURE of the result. GOTO defines the **transitions** of the LR(0) automaton.

Worked Example (Book 4.41)

Let $I = \{ [E' \rightarrow E \cdot], [E \rightarrow E \cdot + T] \}$. Compute $\text{GOTO}(I, +)$:

Only $E \rightarrow E \cdot + T$ qualifies. Move the dot: $E \rightarrow E + \cdot T$. Take CLOSURE:

$\text{GOTO}(I, +) = \{ E \rightarrow E + \cdot T, T \rightarrow \cdot T * F, T \rightarrow \cdot F, F \rightarrow \cdot (E), F \rightarrow \cdot \text{id} \}$

Key Points

- Two steps: (1) shift the dot past X in relevant items, (2) take CLOSURE.
- X can be terminal or non-terminal.
- $\text{GOTO}(I, X)$ is defined (non-empty) only if some item in I has the dot right before X .
- Each edge in the automaton diagram is a GOTO transition labelled X .

Q: Define $\text{GOTO}(I, X)$.

A: CLOSURE of $[A \rightarrow \alpha X \cdot \beta]$ for every $[A \rightarrow \alpha \cdot X \beta]$ in I .

Q: When is $\text{GOTO}(I, X)$ empty?

A: When no item in I has the dot immediately before X .

Common Mistake: Forgetting the CLOSURE step — GOTO is "move the dot, THEN close", not just "move the dot".

Remember: $\text{GOTO}(I, X) = \text{CLOSURE}(\text{shift-the-dot-past-}X \text{ in every applicable item of } I)$.

6. Canonical Collection & DFA Construction

The **canonical LR(0) collection** is the set of all reachable states built by repeatedly applying CLOSURE and GOTO, starting from I_0 .

```
void items(G') {
    C = CLOSURE({[S' -> .S]});
    repeat
        for (each set of items I in C)
            for (each grammar symbol X)
                if (GOTO(I, X) is not empty and not already in C)
                    add GOTO(I, X) to C;
```

```

    until no new sets of items are added to C on a round;
}

```

This is a breadth-first worklist construction of a DFA, starting from $I_0 = \text{CLOSURE}(\{S' \rightarrow \bullet S\})$.

Key Points

- The LR(0) automaton is a DFA whose states are item sets and whose edges are GOTO results.
- **All states are accepting states** — but only reaching $S' \rightarrow S \bullet$ means "accept the whole input".
- Not 100% deterministic in the strict sense: there is no dead state for the empty item set — some (state, input) pairs simply have no next state (signals a syntax error).
- Two different kernel sets that close to identical item sets collapse into the SAME state.

Q: What is the canonical LR(0) collection?

A: All distinct item sets reachable from I_0 by repeatedly applying CLOSURE and GOTO.

Q: Are all states of the LR(0) automaton accepting states?

A: Yes — only the state containing $S' \rightarrow S \bullet$ signals overall acceptance.

Q: Why is the LR(0) automaton not perfectly deterministic in the strict sense?

A: No explicit dead state for the empty item set; some (state, input) pairs have no defined transition — a parse error in practice.

Common Mistake: Creating a duplicate state when GOTO lands back on an existing state — always compare full item-set contents, not just similar-looking labels.

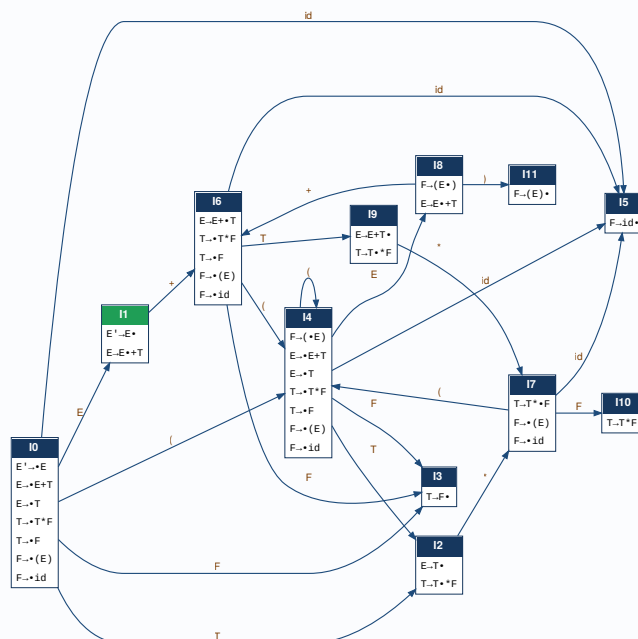
Remember: Canonical collection = keep applying CLOSURE + GOTO from I_0 until no new state appears. Each state = one node; each GOTO = one labelled edge.

7. Worked Example: Expression Grammar (I0–I11)

This is the single most important worked example in the chapter (Book Example 4.40–4.42, Fig. 4.31).

0. $E' \rightarrow E$ (augmented)
1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow \text{id}$

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



Why Each State Looks The Way It Does

- I0: CLOSURE of the augmented item — E, T, F all get their •-productions.
- I1 contains the accept item $E' \rightarrow E\bullet$ — the ONLY state where acceptance can occur.
- I4 looks like I0 because after '(' the parser again expects a full expression before ')'. I3, I5, I10, I11 have NO outgoing transitions — every item is complete, so only reduction is possible.
- I2 and I9 both contain a $T\bullet * F$ item together with a completed item ($E \rightarrow T\bullet$ / $E \rightarrow E+T\bullet$) — these are shift/reduce decision points that plain LR(0) cannot resolve without a full parsing table.

Q: Which state represents "accept"?

A: I1, specifically the item $E' \rightarrow E\bullet$.

Q: Which states have no outgoing transitions, and why?

A: I3, I5, I10, I11 — every item inside them is a completed item.

Q: Why do GOTO(I0,F) and GOTO(I4,F) lead to the same state I3?

A: Because both produce the identical item set $\{T \rightarrow F\bullet\}$ — same item content means same state, regardless of which source state reached it.

Q: What is significant about I2 containing both $E \rightarrow T\bullet$ and $T \rightarrow T\bullet * F$?

A: It is a shift/reduce decision point — shift on '*' to keep building $T*F$, or reduce $E \rightarrow T$ otherwise. LR(0) alone cannot decide; lookahead (SLR/LALR/LR(1)) is needed.

Common Mistake: Missing the accept item $E' \rightarrow E\bullet$ in I1; drawing outgoing arrows from I3/I5/I10/I11 (they are dead ends).

8. Parsing With the Automaton: Stack Trace

An LR parser keeps a stack of automaton **states** $s_0s_1\dots s_m$, with s_m on top.

- **Shift** on input symbol a if state on top has a GOTO transition on a — push the new state.

- **Reduce** otherwise: the completed item in the state tells us which production $A \rightarrow \beta$ to use. Pop $|\beta|$ states, exposing state i ; push $\text{GOTO}(i, A)$.
- **Accept** exactly when about to reduce by $S' \rightarrow S$ with no input left.

Full Trace: Parsing $\text{id} * \text{id}$ (Book Example 4.43)

Line	Stack	Symbols	Remaining Input	Action
1	0	\$	$\text{id} * \text{id} \$$	shift to 5
2	0 5	$\$ \text{id}$	$* \text{id} \$$	reduce by $F \rightarrow \text{id}$
3	0 3	$\$ F$	$* \text{id} \$$	reduce by $T \rightarrow F$
4	0 2	$\$ T$	$* \text{id} \$$	shift to 7
5	0 2 7	$\$ T *$	$\text{id} \$$	shift to 5
6	0 2 7 5	$\$ T * \text{id}$	$\$$	reduce by $F \rightarrow \text{id}$
7	0 2 7 10	$\$ T * F$	$\$$	reduce by $T \rightarrow T * F$
8	0 2	$\$ T$	$\$$	reduce by $E \rightarrow T$
9	0 1	$\$ E$	$\$$	accept

Line 4 — why shift, not reduce: state 2 contains the completed item $E \rightarrow T \bullet$. A naive parser might reduce immediately, but the next symbol is '*', meaning T could still extend into $T * F$. Reducing too early would break the parse of $\text{id} * \text{id}$. Since state 2 has a transition on '*', we shift.

Line 8: now input is just $\$$, and state 2 has no transition on $\$$, so we must reduce $E \rightarrow T$.

Q: What does the LR parser's stack actually hold?

A: A sequence of automaton states (each state, except state 0, corresponds to a unique grammar symbol).

Q: Describe exactly what happens during a reduction by $A \rightarrow \beta$.

A: Pop $|\beta|$ states off the stack, exposing state i ; look up $\text{GOTO}(i, A)$ and push that state.

Q: When does the parser announce accept?

A: When about to reduce by $S' \rightarrow S$ with no input remaining.

Remember: SHIFT: transition exists on next symbol $\rightarrow$ push new state. REDUCE: no transition, but completed item exists $\rightarrow$ pop $|\text{body}|$ states, push $\text{GOTO}(\text{top}, \text{head})$. ACCEPT: about to reduce by $S' \rightarrow S$ at end of input.

9. SLR(1) Parsing Table (Same Grammar)

Building on the same expression-grammar automaton (I0–I11) constructed in Section 7, the SLR(1) ACTION/GOTO table below adds FOLLOW-set lookahead to resolve the shift/reduce choices at states like I2 and I9 that plain LR(0) could not decide on its own.

First and Follow Sets

Production	FIRST	FOLLOW (of LHS)
$E \rightarrow E + T \mid T$	$\text{FIRST}(E) = \{ (, \text{id} \}$	$\text{FOLLOW}(E) = \{ \$, +, , \}$
$T \rightarrow T * F \mid F$	$\text{FIRST}(T) = \{ (, \text{id} \}$	$\text{FOLLOW}(T) = \{ *, \$, +, , \}$
$F \rightarrow (E) \mid \text{id}$	$\text{FIRST}(F) = \{ (, \text{id} \}$	$\text{FOLLOW}(F) = \{ *, \$, +, , \}$

SLR(1) Parsing Table

State	id	+	*	(	)	\$	E	T	F
0	S5			S4			1	2	3
1		S6				Accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Moves of SLR(1) Parser on id * id

Line	Stack	Symbol	Input	Action
1	0	\$	id * id \$	Shift to 5
2	0 5	\$ id	* id \$	Reduce F → id
3	0 3	\$ F	* id \$	Reduce T → F
4	0 2	\$ T	* id \$	Shift to 7
5	0 2 7	\$ T *	id \$	Shift to 5
6	0 2 7 5	\$ T * id	\$	Reduce F → id
7	0 2 7 10	\$ T * F	\$	Reduce T → T * F
8	0 2	\$ T	\$	Reduce E → T
9	0 1	\$ E	\$	Accept

Note: the plain LR(0) table (without FOLLOW-set lookahead) would show conflicting entries at states 2 and 9 for both "*" and "\$"/")" — that is exactly why SLR adds FOLLOW sets to resolve them.

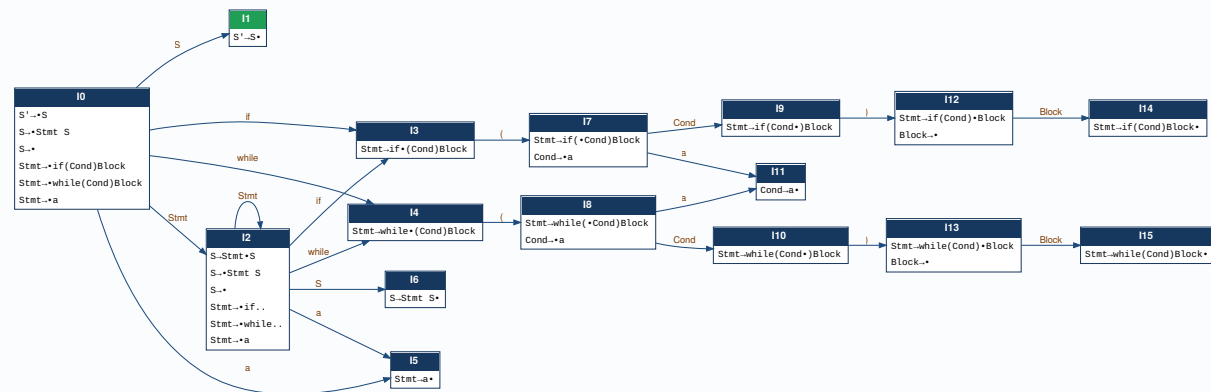
10. Simulation: If / While Statement Grammar

Grammar:

1. $S \rightarrow \text{Stmt } S$
2. $S \rightarrow \epsilon$
3. $\text{Stmt} \rightarrow \text{if (Cond) Block}$
4. $\text{Stmt} \rightarrow \text{while (Cond) Block}$
5. $\text{Stmt} \rightarrow a$
6. $\text{Block} \rightarrow \epsilon$
7. $\text{Cond} \rightarrow a$

Augmented: $\emptyset. S' \rightarrow S$

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



Why Each Step Happens

- I0 pulls in Stmt's productions (dot before Stmt), which further pulls in if/while/a.
- I0 ALSO contains $S \rightarrow \cdot$ (from $S \rightarrow \epsilon$) directly — an immediately-completed item the moment it is created, since ϵ has zero symbols.
- I2 is structurally identical to I0 because after one Stmt, the grammar again expects a full S (S is self-recursive).
- I7 and I8 both pull in $\text{Cond} \rightarrow \cdot a$ via closure, since both $\text{if}(\dots)$ and $\text{while}(\dots)$ expect a Cond after '('.
- I5, I6, I11, I14, I15 have no outgoing transitions — pure reduce states.

Q: Why do I0 and I2 support an immediate reduction by $S \rightarrow \epsilon$?

A: Because $S \rightarrow \epsilon$ produces the already-completed item $S \rightarrow \cdot$ the instant its closure is triggered, sitting right alongside the shiftable Stmt/if/while/a items.

Q: Does Block $\rightarrow \epsilon$ behave the same way?

A: Yes — it makes $\text{Block} \rightarrow \cdot a$ a completed item in I12 and I13 as soon as those states form.

Common Mistake: Forgetting that $S \rightarrow \epsilon$ and $\text{Block} \rightarrow \epsilon$ each silently contribute an already-completed item wherever their closure is triggered.

Remember: Every ϵ -production silently contributes an already-completed item ($X \rightarrow \cdot$) the moment its closure is triggered.

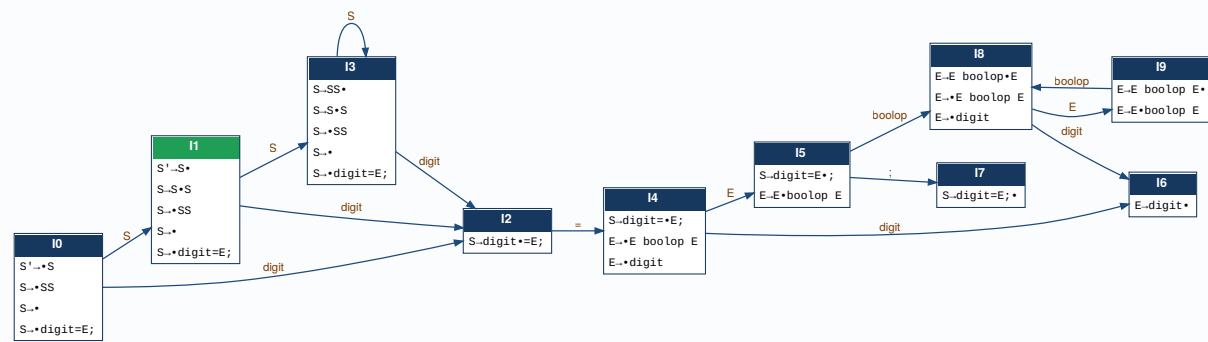
11. Simulation: $S \rightarrow SS$ Grammar (Not LR(0))

Grammar:

1. $S \rightarrow S S$
2. $S \rightarrow \epsilon$
3. $S \rightarrow \text{digit} = E ;$
4. $E \rightarrow E \text{ boolop } E$
5. $E \rightarrow \text{digit}$

Augmented: $\emptyset. S' \rightarrow S$

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



Why This Grammar Is NOT LR(0)

This grammar has several shift/reduce and reduce/reduce conflicts, caused by $S \rightarrow \epsilon$ and the left-recursive $E \rightarrow E \text{ boolop } E$.

- **Conflict 1 ($S \rightarrow \epsilon$):** In I0 and I1, the completed item $S \rightarrow \cdot$ sits beside items like $S \rightarrow \cdot SS$ and $S \rightarrow \cdot \text{digit}=E$; that still expect more input. The parser cannot tell — without lookahead — whether to reduce immediately or keep shifting.
- **Conflict 2 (left recursion):** In I9, the completed item $E \rightarrow E \text{ boolop } E \cdot$ sits next to the incomplete item $E \rightarrow E \cdot \text{boolop } E$. On seeing another boolop, LR(0) cannot decide whether to reduce now or shift and keep extending.

Important: the LR(0) automaton can still be built for any grammar — CLOSURE and GOTO do not require conflict-freeness. What fails is the LR(0) parsing table, when two actions are legally required in the same state with no lookahead to distinguish them.

Q: Is $S \rightarrow SS \mid \epsilon \mid \text{digit}=E$; an LR(0) grammar?

A: No. Shift/reduce conflicts occur where $S \rightarrow \cdot$ sits with items expecting more input (caused by $S \rightarrow \epsilon$), and where the left-recursive $E \rightarrow E \text{ boolop } E$ rule creates conflicting completed/incomplete items.

Q: Can an LR(0) automaton be built even for a non-LR(0) grammar?

A: Yes — CLOSURE/GOTO always apply; conflicts only surface when converting to an ACTION/GOTO table.

Q: Difference between shift/reduce and reduce/reduce conflict?

A: Shift/reduce = one completed item + one shiftable item on the same symbol in the same state. Reduce/reduce = two or more completed items (different productions) in the same state.

Remember: $S \rightarrow \epsilon$ and ambiguous left-recursive rules (like $E \rightarrow E \text{ boolop } E$) are classic causes of LR(0) conflicts. Automaton construction $\neq$ conflict-free parsing table — two independent facts.

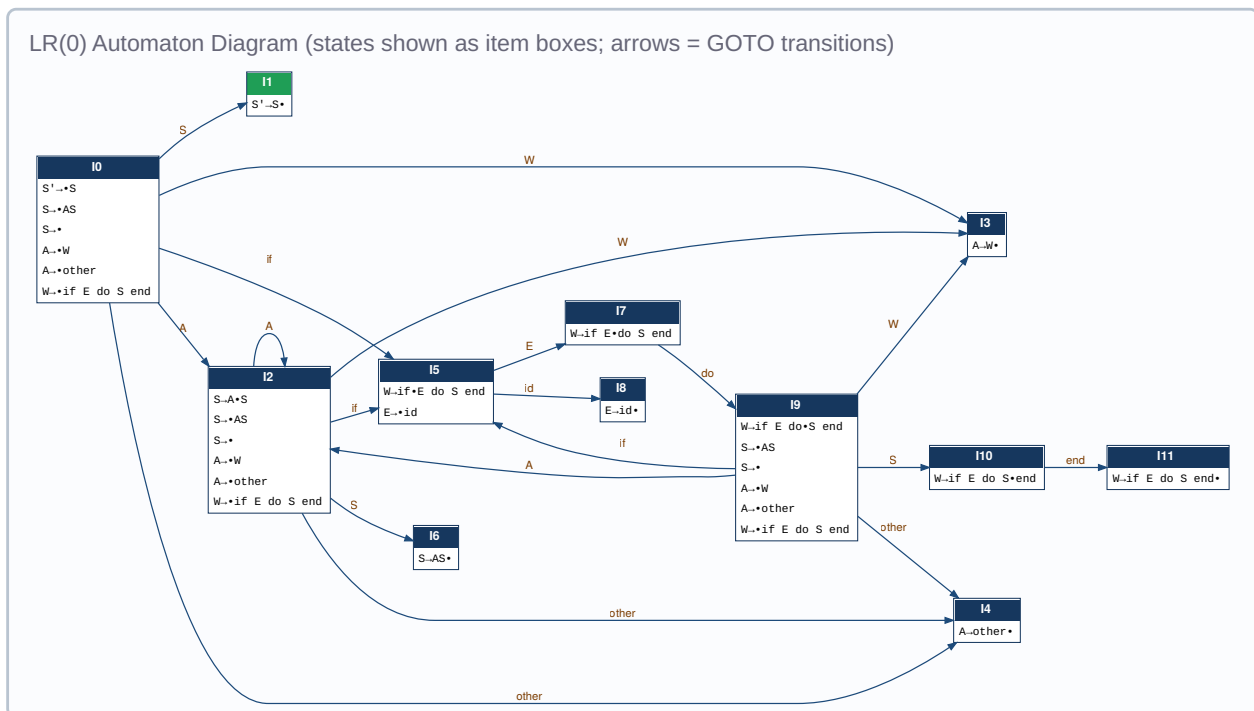
12. Simulation: if-E-do-S-end Grammar

Grammar:

1. $S \rightarrow A \ S$
2. $S \rightarrow \epsilon$
3. $A \rightarrow W$
4. $A \rightarrow \text{other}$
5. $W \rightarrow \text{if } E \text{ do } S \text{ end}$

6. $E \rightarrow id$

Augmented: $0. S' \rightarrow S$



Why the Same S/A/W Closure Block Repeats Three Times

I0, I2, and I9 all contain the identical block $\{S \rightarrow \bullet AS, S \rightarrow \bullet, A \rightarrow \bullet W, A \rightarrow \bullet \text{other}, W \rightarrow \bullet \text{if } E \text{ do } S \text{ end}\}$. This is expected, not an error — a full statement list S is expected in three contexts: program start (I0), right after a statement A (I2, since $S \rightarrow AS$ is recursive), and right after 'do' inside an if-block (I9). Closure only depends on *which non-terminal follows the dot*, not on the path taken to reach the state — so identical closure blocks legitimately reappear, though I0, I2, I9 remain distinct states because their kernel items differ.

Q: Why does closure at I9 re-derive the entire S/A/W block, just like I0?

A: Because $W \rightarrow$ if E do $\bullet S$ end has its dot right before S , triggering the same closure rule as $S' \rightarrow \bullet S$ in I_0 .

Q: List the pure-reduce states in this automaton.

A: I1 (accept), I3 ($A \rightarrow W\bullet$), I4 ($A \rightarrow \text{other}\bullet$), I6 ($S \rightarrow AS\bullet$), I8 ($E \rightarrow \text{id}\bullet$), I11 ($W \rightarrow \text{if } E \text{ do } S \text{ end}\bullet$).

Common Mistake: Assuming I0, I2, I9 must be different states just because reached differently — always compare item contents; here they are legitimately distinct because their kernel items differ ($S' \rightarrow \bullet S$ vs $S \rightarrow A \bullet S$ vs $W \rightarrow \text{if } E \text{ do } \bullet S \text{ end}$).

Remember: Closure is purely local — it depends only on which non-terminal sits right after the dot, so the same closure block can reappear in multiple, otherwise-different states.

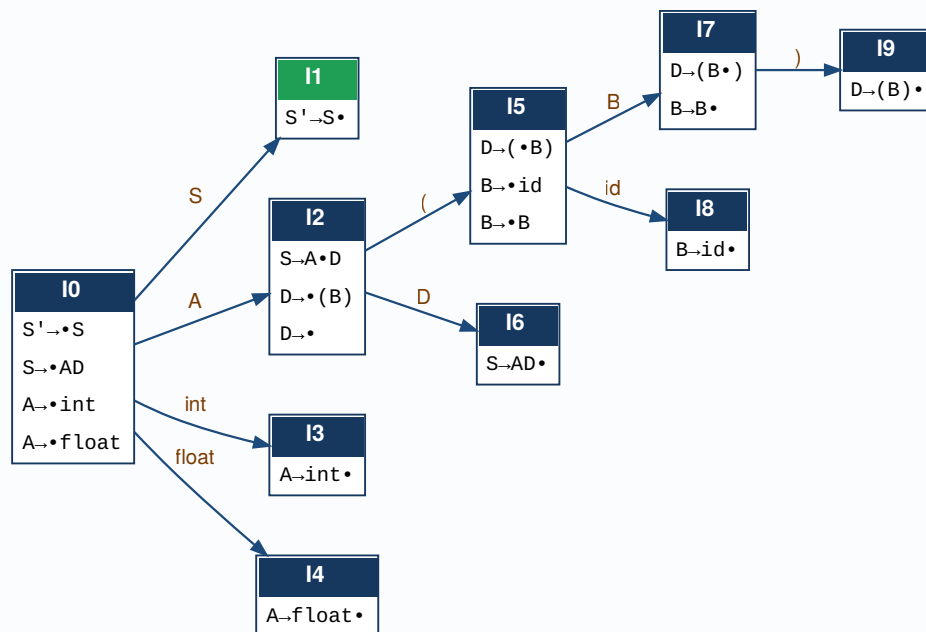
13. Simulation: $S \rightarrow A D$ Grammar (Shift/Reduce Conflict)

Grammar :

$$1. S \rightarrow A D$$

2. $A \rightarrow \text{int}$
 3. $A \rightarrow \text{float}$
 4. $D \rightarrow (B)$
 5. $D \rightarrow \epsilon$
 6. $B \rightarrow \text{id}$
 7. $B \rightarrow B$
 Augmented: 0. $S' \rightarrow S$

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



The Shift/Reduce Conflict at I7

State $I7 = \{D \rightarrow (B\bullet), B \rightarrow B\bullet\}$ contains both:

- $D \rightarrow (B\bullet)$ — shiftable: on seeing ')' next, shift into I9.
- $B \rightarrow B\bullet$ — a completed item: we could reduce by $B \rightarrow B$ right now, before even seeing ')

Both a shift (on ')') and a reduce ($B \rightarrow B$) are simultaneously legal, and LR(0) has no lookahead to choose — a textbook shift/reduce conflict. The cause is the unusual production $B \rightarrow B$, which is directly left-recursive with a right side identical to its own left side.

Q: Where exactly does the conflict occur, and why?

A: In $I7 = \{D \rightarrow (B\bullet), B \rightarrow B\bullet\}$: on ')' the parser could shift (toward $D \rightarrow (B)\bullet$) or reduce by $B \rightarrow B$, with no way to decide under LR(0).

Q: What is unusual about $B \rightarrow B$?

A: Its right side equals its left side — a production that can never make forward progress alone, guaranteeing a completed item sits beside a still-shiftable item.

Exam Tip: Whenever you see a production of the exact form $X \rightarrow X$, immediately flag it as a shift/reduce conflict candidate.

Remember: A production $X \rightarrow X$ is a guaranteed shift/reduce trigger: the moment X is recognised, $X \rightarrow X\bullet$ is already complete, sitting beside whatever else was expecting X to continue.

14. Quiz Practice Examples

The following four grammars are additional practice examples (Quiz 02 style) solved in full using the same CLOSURE/GOTO method as above.

14.1 Declaration Grammar: $s \rightarrow a\ c$

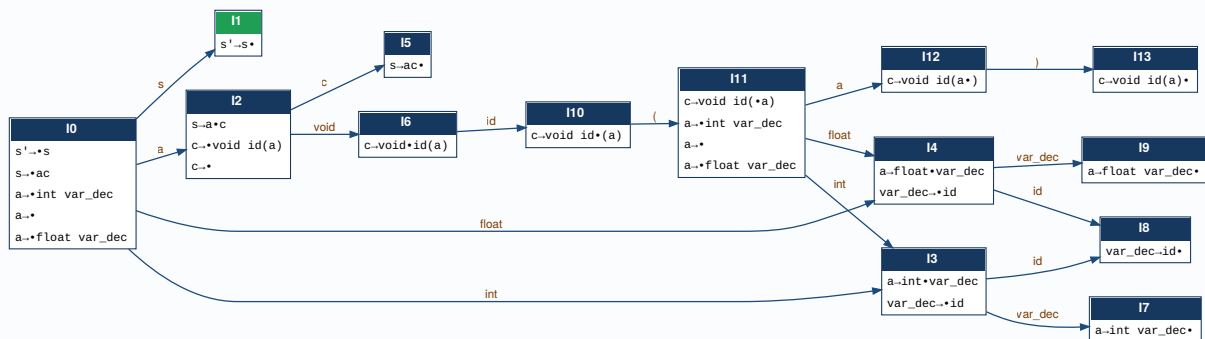
Grammar:

$s \rightarrow a\ c$
 $a \rightarrow \text{int var_dec} \mid \varepsilon \mid \text{float var_dec}$
 $\text{var_dec} \rightarrow \text{id}$
 $c \rightarrow \text{void id (a)} \mid \varepsilon$

Augmented & numbered:

0. $s' \rightarrow s$
1. $s \rightarrow a\ c$
2. $a \rightarrow \text{int var_dec}$
3. $a \rightarrow \varepsilon$
4. $a \rightarrow \text{float var_dec}$
5. $\text{var_dec} \rightarrow \text{id}$
6. $c \rightarrow \text{void id (a)}$
7. $c \rightarrow \varepsilon$

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



Why I3 and I4 get reused at I11: the closure of $c \rightarrow \text{void id}(\cdot a)$ needs all of a 's productions again — exactly the same item sets already built as I3 and I4. Since $\text{GOTO}(I11, \text{int})$ and $\text{GOTO}(I11, \text{float})$ produce item sets identical to I3 and I4, the transitions point back to those existing states instead of creating duplicates.

Q: Why does $a \rightarrow \varepsilon$ make I0 and I11 support an immediate reduce?

A: Because $a \rightarrow \varepsilon$ contributes the completed item $a \rightarrow \cdot$ the instant closure adds a 's productions. This happens only in I0 and I11 — the two states whose closure pulls in a 's productions (dot before non-terminal a).

Q: Why are I3 and I4 reached again from I11 instead of new states being built?

A: $\text{GOTO}(I11, \text{int})$ and $\text{GOTO}(I11, \text{float})$ produce item sets with exactly the same contents as I3 and I4 respectively — states are identified by item-set content, not by how they were reached.

14.2 Statement Grammar: $\text{expr} \rightarrow \text{while}(\text{cond}) \mid \text{epsilon} \mid \text{decl}$

Grammar:

$\text{expr} \rightarrow \text{while} (\text{cond}) \mid \text{epsilon} \mid \text{decl}$

$\text{cond} \rightarrow \text{var op var c}$

$\text{decl} \rightarrow \text{char C}$

$C \rightarrow \text{var}$

Augmented & numbered:

0. $\text{expr}' \rightarrow \text{expr}$

1. $\text{expr} \rightarrow \text{while} (\text{cond})$

2. $\text{expr} \rightarrow \text{epsilon}$

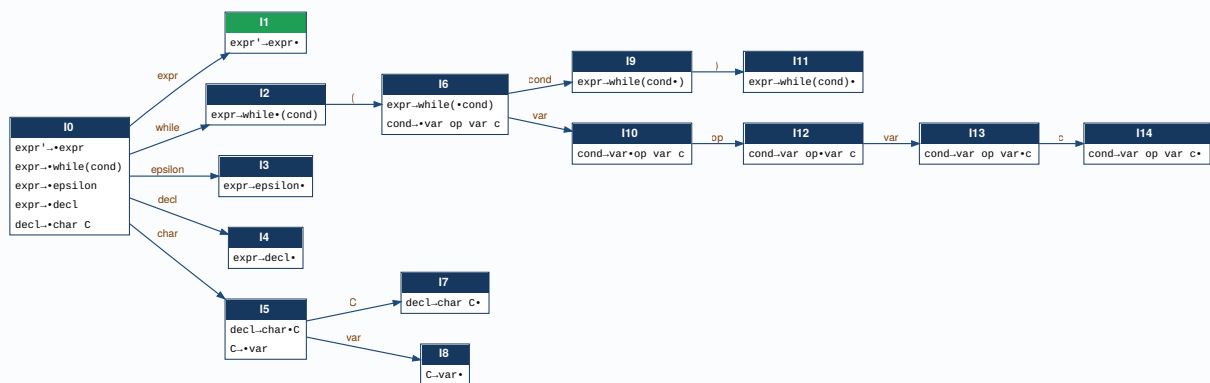
3. $\text{expr} \rightarrow \text{decl}$

4. $\text{cond} \rightarrow \text{var op var c}$

5. $\text{decl} \rightarrow \text{char C}$

6. $C \rightarrow \text{var}$

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



Q: Why does I0 close in $\text{decl} \rightarrow \bullet \text{char C}$ directly?

A: Because $\text{expr} \rightarrow \bullet \text{decl}$ has the dot before non-terminal decl, so closure pulls in all of decl's productions too.

Q: What kind of state is I11 ($\text{expr} \rightarrow \text{while}(\text{cond})\bullet$)?

A: A pure reduce state — after reducing, $\text{GOTO}(\text{I0}, \text{expr})$ sends control to I1, the accept state, if input is exhausted.

14.3 Ambiguous Grammar: $S \rightarrow \text{SSS} + \mid \text{SS} * \mid a \mid == = \mid e$

Grammar: $S \rightarrow \text{SSS} + \mid \text{SS} * \mid a \mid == = \mid e$

Augmented & numbered:

0. $S' \rightarrow S$

1. $S \rightarrow \text{SSS} +$

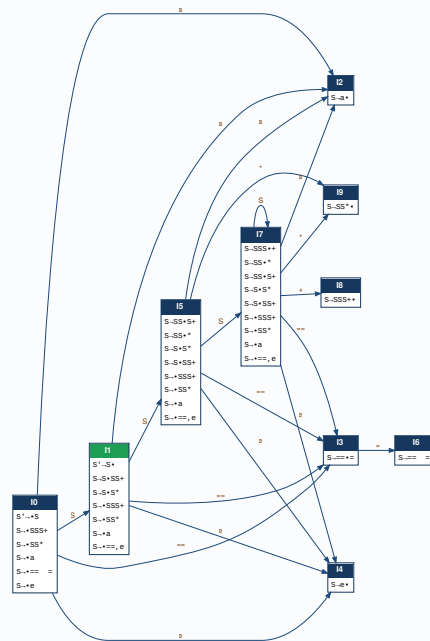
2. $S \rightarrow \text{SS} *$

3. $S \rightarrow a$

4. $S \rightarrow == =$ (two terminals: "==" then "=")

5. $S \rightarrow e$

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



States I2, I3, I4 (the reduce states for a, ==, e) are reached from every state that has $S/a/==/e$ right after the dot — I0, I1, I5, I7 — and because the resulting item set is identical each time, they all correctly point to the SAME shared states rather than duplicating them.

Because $S \rightarrow S S S +$ and $S \rightarrow S S *$ both start by expecting more S's, state I7 (which has already seen SS or SSS) also has a transition on S back to **itself** — a self-loop. This reflects the unbounded left recursion: the parser can keep shifting additional S's while still deciding whether the eventual production will be SSS+ or SS*.

Q: Why do I0, I1, I5, I7 all transition to the same I2, I3, I4?

A: Because GOTO on a, ==, or e from any of these states produces an identical single-item completed set each time (e.g. $\{S \rightarrow a \cdot\}$), so they collapse into shared states.

Q: Why does I7 have a self-loop on S?

A: Because I7's items (from $S \rightarrow SSS \cdot +$, $S \rightarrow SS \cdot S+$, $S \rightarrow S \cdot SS+$, $S \rightarrow S \cdot S*$, plus the closure items) still expect further S's before the eventual + or * is seen, so shifting another S keeps the parser inside the same item set.

14.4 Record Grammar: $p_t \rightarrow \text{record } \{ \text{def } \} \mid \epsilon$

Grammar:

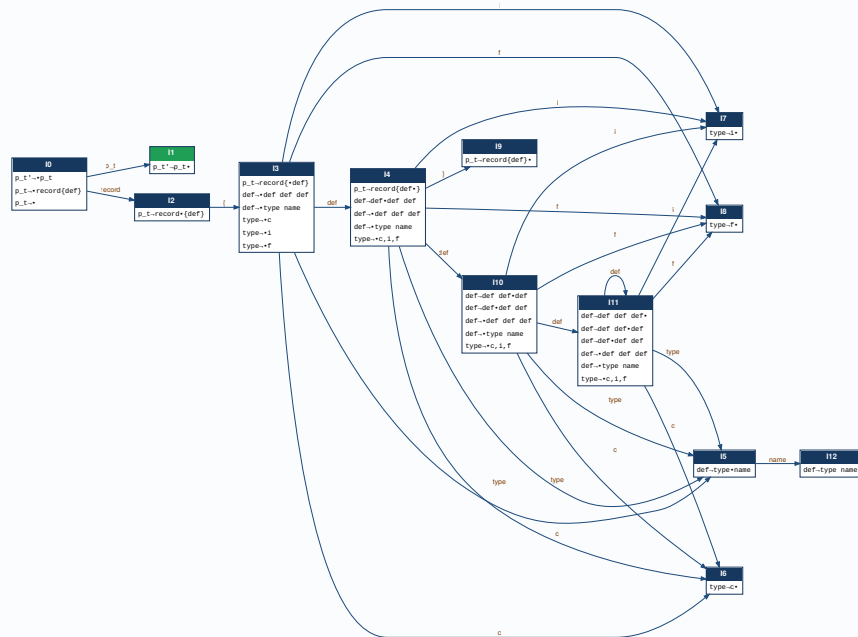
```
p_t → record { def } | ε
def → def def def | type name
type → c | i | f
```

Augmented & numbered:

0. $p_t' \rightarrow p_t$
1. $p_t \rightarrow \text{record } \{ \text{def } \}$
2. $p_t \rightarrow \epsilon$
3. $\text{def} \rightarrow \text{def def def}$
4. $\text{def} \rightarrow \text{type name}$
5. $\text{type} \rightarrow c$

6. type $\rightarrow$ i
7. type $\rightarrow$ f

LR(0) Automaton Diagram (states shown as item boxes; arrows = GOTO transitions)



Why I5, I6, I7, I8 keep reappearing: the closure block for "type" ($\text{type} \rightarrow c \mid i \mid f$) is triggered independently at I3, I4, I10, and I11 — every context where a "def" is expected. Since the resulting item sets are identical every time, GOTO reuses the same states I5–I8 instead of duplicating them, exactly like the repeated closure pattern seen in Section 12 (if–E–do–S–end grammar).

The shift/reduce conflict at I11: state $I11 = \{\text{def} \rightarrow \text{def} \text{ def} \text{ def} \bullet, \text{def} \rightarrow \text{def} \text{ def} \text{ def} \bullet \text{def}, \text{def} \rightarrow \text{def} \text{ def} \text{ def} \bullet \text{type} \text{ name}, \text{type} \rightarrow c \mid i \mid f\}$ contains the completed item $\text{def} \rightarrow \text{def} \text{ def} \text{ def} \bullet$ *together with* still-open items that expect another def. If a fourth "def" follows, the parser could either reduce the first three def's into one def right now, or keep shifting to build a longer chain — both are legal in the same state with no lookahead to decide, so I11 self-loops on def and is a genuine shift/reduce conflict, caused by the self-referencing production $\text{def} \rightarrow \text{def} \text{ def} \text{ def}$ (similar in spirit to the $B \rightarrow B$ conflict in Section 13).

Q: Why does $p_t \rightarrow \epsilon$ not create a separate labelled reduce state in I0?

A: It does — $p_t \rightarrow \bullet$ is already a completed item inside I0 the moment closure adds it, so I0 itself supports an immediate reduce by $p_t \rightarrow \epsilon$ alongside the shift on "record".

Q: Why are I5–I8 reused across I3, I4, I10, and I11?

A: Because each of these states independently closes the same non-terminal ("def" expecting a type), producing identical item sets — states are merged whenever their contents match exactly.

Q: Where exactly is the shift/reduce conflict in this grammar, and why?

A: At I11, which contains the completed item $\text{def} \rightarrow \text{def} \text{ def} \text{ def} \bullet$ alongside items like $\text{def} \rightarrow \text{def} \text{ def} \text{ def} \bullet \text{def}$ that still expect another def — on seeing a further "def", LR(0) cannot decide whether to reduce immediately or keep shifting.